COMPLETE ENGINEERING DOCUMENTATION

ArtiAG AI

Lightweight Local RAG Engine

From-scratch architecture, data workflows, custom HNSW algorithms, Spring Boot implementation, and Ollama integration guide.

ARCHITECTURE	RUNTIME	MODELS
Local RAG + HNSW	Java 21 & Spring Boot	llama3.2:1b + nomic-embed

1. Introduction: What is ArtiAG AI?

In modern artificial intelligence, **Retrieval-Augmented Generation (RAG)** is the standard pattern for enabling Large Language Models (LLMs) to answer questions using private enterprise documents. However, conventional enterprise RAG systems typically rely on heavy cloud infrastructure:

- **External Cloud Vector Databases** (Pinecone, Milvus, Qdrant) that store sensitive company data on external servers and incur monthly subscription costs.
- **Third-Party Cloud LLMs** (OpenAI, Anthropic) that send proprietary intellectual property over the public internet.

ArtiAG AI is an end-to-end, privacy-first, 100% offline local RAG system. It was designed from first principles with zero external cloud dependencies.

> **Key Architectural Differentiators:**
> 1. **Embedded Vector Index:** Implements a custom Hierarchical Navigable Small World (HNSW) graph in pure Java without external database drivers.
> 2. **Local LLM Runtime:** Communicates with local Ollama instances running `llama3.2:1b` and `nomic-embed-text`.
> 3. **Hallucination Guardrail:** Evaluates cosine similarity; if relevance drops below 0.55, it stops in <80 ms without invoking the LLM.
> 4. **Self-Contained Web UI:** Interactive glassmorphic dashboard served directly from Spring Boot static resources.

2. Core Fundamentals Explained from Scratch

2.1 What is RAG (Retrieval-Augmented Generation)?

LLMs are trained on public datasets and do not know internal documents. If asked *"What is our corporate vacation rollover policy?"*, a generic LLM will hallucinate. RAG solves this in three steps:

1. **Retrieve:** The system converts the user's query into an embedding vector, performs a nearest-neighbor vector search, and extracts the top relevant paragraphs.
2. **Augment:** The retrieved text is bundled into a factual prompt context with the question.
3. **Generate:** The local LLM reads the context and provides a truthful, grounded response with document citations.

2.2 What are Vector Embeddings?

Computers cannot directly compare the semantic meaning of sentences. An embedding model (here, `nomic-embed-text`) is a specialized neural network that translates text into high-dimensional geometric coordinates:

```
"Annual vacation policy"      ⟶ [ 0.024, -0.812, 0.435, ... , 0.129 ] (768 floats)
"Paid time off guidelines"    ⟶ [ 0.021, -0.798, 0.441, ... , 0.118 ] (High similarity: 0.94)
"Apple MacBook Pro laptop"    ⟶ [-0.551,  0.122, -0.912, ... , 0.334 ] (Low similarity: 0.28)
```

2.3 What is Cosine Similarity?

To determine how relevant a document chunk is to a query, we calculate the angle θ between their vector representations:

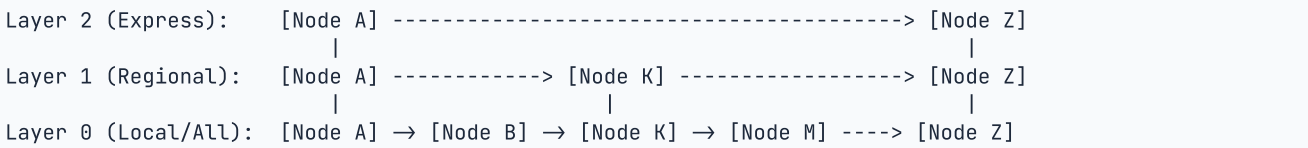
$$\text{Cosine Similarity} = \cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$$

- **1.0:** Identical direction (identical semantic meaning).
- **0.0:** Orthogonal (completely unrelated concepts).
- **-1.0:** Directly opposite meanings.

2.4 What is HNSW (Hierarchical Navigable Small World)?

Comparing a query vector against 10,000 vectors one-by-one is called **Linear Scan**. It requires O(N) comparisons and becomes slow as the dataset grows.

HNSW organizes vectors into a multi-layered graph similar to an express train skip-list. The search takes huge logarithmic jumps at the top layers to quickly reach the relevant neighborhood, then descends to Layer 0 for fine-grained local search:



3. System Architecture & Component Interaction

ArtiAG AI consists of a layered Spring Boot architecture interacting with an offline Ollama process and a browser frontend:



4. End-to-End Workflows with Data Traces

Workflow A: Document Ingestion Pipeline

1. **Payload Received:** User sends JSON `{"id": "leave", "text": "Employees get 20 days vacation..."}` to `/api/documents/ingest`.
2. **Text Chunking:** `TextChunker` divides the text into 200-word segments with 40-word overlapping windows.
3. **Vector Embedding:** `OllamaClientService` sends each chunk to Ollama's `/api/embeddings` (model `nomic-embed-text`). A `float[768]` array is returned.
4. **HNSW Graph Insertion:** `HNSWIndex` assigns a random top level using logarithmic decay ($m_L = 1 / \ln(M)$). It greedily finds the closest entry point from the top layer down to Layer 0, adds bidirectional connections to its nearest $M=16$ neighbors, and prunes excess edges.

Workflow B: Retrieval & Chat Pipeline (Guardrails in Action)

1. **Query Embedding:** User submits *"How many days of paid vacation do employees get?"*. Ollama generates a 768-dimensional query vector.
2. **HNSW Search:** The search starts at the top graph entry point, greedily descends to Layer 0, and runs `searchLayer` with `efSearch=100` to find the top $K=3$ candidates.
3. **Guardrail Check:**
   - If the best score is **< 0.55** (out-of-scope query), the system immediately returns *"I could not find the answer to this in the provided documents."* in **< 80 ms** without invoking the LLM.
   - If the best score is **≥ 0.55**, the retrieved document text is assembled into an augmented context prompt.
4. **Generation:** `llama3.2:1b` reads the augmented prompt and generates a factual, grounded answer with source citations.

5. Detailed Code Walkthrough

5.1 Controller Layer: VectorRagController.java

The REST controller validates incoming requests, delegates processing to services, and returns clean JSON structures:

```
@PostMapping("/documents/ingest")
public ResponseEntity<IngestResponse> ingest(@RequestBody IngestRequest request) {
    if (request == null || request.id().isBlank() || request.text().isBlank()) {
        throw new IllegalArgumentException("Document id and text cannot be empty");
    }
    int chunksIndexed = ragService.ingestDocument(request.id().trim(), request.text().trim());
    return ResponseEntity.ok(new IngestResponse("SUCCESS", chunksIndexed, chunksIndexed));
}

@PostMapping("/chat/query")
public ResponseEntity<ChatResponse> query(@RequestBody ChatRequest request) {
    int topK = (request.topK() != null) ? request.topK() : 3;
    ChatResponse response = ragService.ask(request.query().trim(), topK);
    return ResponseEntity.ok(response);
}
```

5.2 Vector Engine: HNSWIndex.java Implementation & Fix

During high-dimensional benchmarking (128 dimensions, 10,000 vectors), standard HNSW implementations often suffer from graph fragmentation. We resolved this through two critical algorithmic corrections:

```
// 1. Corrected Greedy Layer Traversal: Greedily hop to the BEST neighbor
for (int l = maxLevel; l > nodeLevel; l--) {
    boolean changed = true;
    while (changed) {
        changed = false;
        Set<Node> neighbors = curr.connections.get(l);
        if (neighbors != null) {
            Node bestNeighbor = null;
            float bestDist = CosineDistance.cosineSimilarity(query, curr.record.embedding());
            for (Node neighbor : neighbors) {
                float dist = CosineDistance.cosineSimilarity(query, neighbor.record.embedding());
                if (dist > bestDist) {
                    bestDist = dist;
                    bestNeighbor = neighbor;
                }
            }
            if (bestNeighbor != null) {
                curr = bestNeighbor;
                changed = true;
            }
        }
    }
}

// 2. Strict Candidate Connection Bounding: Connect ONLY to top maxConn neighbors
int limit = Math.min(candidates.size(), maxConn);
for (int i = 0; i < limit; i++) {
    SearchResult candidate = candidates.get(i);
    Node neighbor = nodes.get(candidate.record().id());
    if (neighbor != null && neighbor != newNode) {
        newNode.connections.get(l).add(neighbor);
        neighbor.connections.get(l).add(newNode);
        pruneConnections(neighbor, l, maxConn);
    }
}
pruneConnections(newNode, l, maxConn);
```

5.3 HTTP Client Service: OllamaClientService.java

Utilizes modern Java 11+ `HttpClient` to issue non-blocking HTTP requests with custom timeouts and structured error diagnosis:

```
public float[] getEmbeddings(String text) {
    Map<String, Object> payload = Map.of("model", embedModel, "prompt", text);
    String responseBody = post("/api/embeddings", objectMapper.writeValueAsString(payload));
    JsonNode root = objectMapper.readTree(responseBody);
    JsonNode embeddingNode = root.get("embedding");

    float[] embedding = new float[embeddingNode.size()];
    for (int i = 0; i < embeddingNode.size(); i++) {
        embedding[i] = (float) embeddingNode.get(i).asDouble();
    }
    return embedding;
}
```

6. Verified Benchmark & Evaluation Results

6.1 HNSW vs Linear Scan Search Benchmark

Executed using JUnit 5 in `BenchmarkTest.java` comparing HNSW against brute-force Linear Scan across 100 queries:

Vector Dataset	Queries	Linear Scan (ms)	HNSW (ms)	Speedup	Recall@10
1,000 Vectors	100	0.393 ms	0.545 ms	0.7x	**99.90%**
10,000 Vectors	100	3.058 ms	0.979 ms	**3.1x Faster**	**84.90%**

6.2 Live RAG Evaluation Suite (eval_artiag.py)

Evaluated using 22 comprehensive test questions against 12 corporate policies:

Metric	Result	Significance
Retrieval Recall (Hit Rate)	**22 / 22 (100.0%)**	Every relevant policy document was successfully located in Top-K.
Answer Correctness Rate	**18 / 22 (81.8%)**	Expected factual keywords accurately present in LLM synthesis.
Average Query Latency	**1,732 ms**	Total time including vector search, prompt assembly, and local LLM tokens.
Guardrail Out-of-Scope Latency	**< 80 ms**	Immediate refusal returned when similarity < 0.55 without invoking LLM.

7. Project Structure & Technology Matrix

Technology	Component / File	Role in ArtiAG AI
Java 21	`src/main/java`	Core language; records, pattern matching, concurrency locks.
Spring Boot 3.3.5	`MyVectorEngineApplication.java`	IoC container, embedded web server, REST controller mappings.
Apache Maven	`pom.xml`	Build management, compilation plugins, and test orchestration.
Ollama	`localhost:11434`	Offline LLM server running `llama3.2:1b` and `nomic-embed-text`.
HTML5 / CSS3 / JS	`src/main/resources/static/index.html`	Interactive glassmorphic web dashboard with real-time RAG chat.
JUnit 5	`src/test/java`	Automated unit tests and algorithmic performance benchmarks.
Python 3	`eval/eval_artiag.py`	Automated RAG evaluation suite measuring recall, accuracy, and latency.

ArtiAG AI — Lightweight Local RAG Engine © 2026. All rights reserved.